

Univerzita Karlova
Prírodovedecká fakulta



RLE kompresia

Dokumentácia k programu na predmet:
Úvod do programovania (MZ370P19)

Anna Fajtová
2. B-SGG

Vavrečka 2026

Obsah

Úvod a stručné zadanie	3
Presné zadanie	3
Zvolený algoritmus	3
Diskusia výberu algoritmu	3
Program	4
Problematické situácie a ich ošetrenia	5
Alternatívne riešenia úlohy	5
Vstupné dáta	5
Výstupné dáta	6
Priebeh práce	6
Námety na vylepšenie	6
Záverečný povzdych	6

Úvod a stručné zadanie

Dokumentácia prezentuje priebeh a detaily práce na programe pre implementovanie RLE (Run-Length Encoding) kompresie. Program je tvorený v programovacom jazyku Python a využíva objektovo orientované programovanie (OOP). RLE spočíva v tom, že dlhé sekvencie rovnakých hodnôt sa nahradia dvojicou čísel tvorenou počtom opakovaní hodnoty a následne samotnou hodnotou s cieľom efektívneho ukladania postupností opakujúcich sa čísel. Vstupné dáta sa najprv skomprimujú s využitím escape znaku a následne sa pre kontrolu dekomprimujú do pôvodnej podoby. Všetko sa to spoločne uloží do výsledného textového súboru.

Presné zadanie

Pre zadanú postupnosť vstupných dát predstavovanú celými číslami načítanými z textového súboru implementujte metódu RLE kompresie. Ako escape znak použite hodnotu 255, ktorú zvýšite o počet opakovaní. Napíšte aj skript, ktorý spraví dekompresiu súboru. Výsledky uložte do textového súboru.

Zvolený algoritmus

Pri riešení úlohy som zvolila algoritmus RLE vo verzii, ktorá pracuje s fixnými dvojicami dát. Algoritmus najprv prejde vstupnú postupnosť, identifikuje súvislé úseky rovnakých čísel a následne každý úsek nahradí dvojicou zloženou z escape znaku sčítaného s počtom opakovaní čísla a samotného čísla. Algoritmus dekompresie potom číta skomprimované dáta po vytvorených dvojiciach – od prvého znaku odčíta 255 a získa počet opakovaní a potom použije tento počet na počet zapísaní druhého znaku.

Diskusia výberu algoritmu

Pri návrhu algoritmu som najprv zvažovala verziu, kde výskyt escape znaku je závislý na počte výskytov, a teda ak sa číslo vyskytuje iba raz, bude zapísané bez escape znaku. Varianta by bola úspornejšia, ale nakoniec som ju zavrhla, keďže by to spôsobilo problémy pri výskyte čísel vyšších ako 254 v programe – celkovo by sa v tom prípade výstup stal chaotickým, nečitateľným a predovšetkým by nastal chaos keď by sa to program snažil späťne dekomprimovať. Nakoniec som teda zvolila variantu, kde sa vytvoria fixné dvojice escape znaku a daného čísla. Zvolená verzia je oveľa robustnejšia, algoritmus je rýchlejší, eliminuje riziko pomýlenia si dát s escape znakom, a teda je to aj oveľa jednoduchšie využiteľné pri dekompresii.

Program

Program je robený ako modulárny systém, kde je celá výpočetná logika odčlenená od zvyšku a zastrešená pod triedou *RLE*. V triede *RLE* sa nachádza jej konštruktor `__init__`, ktorý inicializuje privátny atribút `__escape`. Hlavnú funkčnosť programu zaisťujú dve metódy – *compress* a *decompress*. Metóda *compress* transformuje vstupný zoznam na komprimovanú verziu. Pre zistenie sekvencie rovnakých čísel používa cyklus *while*, ktorý vytvorí dvojice s escape znakom 255 podľa RLE princípu. Metóda *decompress* potom slúži ako inverzná operácia pre kontrolu. Používa funkciu `range(0, len(data), 2)`, ktorá umožňuje postupne skákať cez komprimovaný zoznam po dvojiciach a správne ich rekonštruovať späť. Ako jednu z hlavných dátových štruktúr program využíva dynamické zoznamy pre ukladanie vstupných dát, komprimovaných dvojíc aj dekomprimovaného výsledku.

Ďalej nasleduje hlavný blok kódu, v ktorom sa dejú rôzne kontroly vstupných dát a celkovo sa zabezpečuje plynulý chod validných informácií od načítania až po záverečný zápis do nového textového súboru.

```
1  #RLE kompresia
2  #Anna Fajtová, 2. ročník, B-SGG
3  #zimný semester 2025/2026
4  #Úvod do programovania MZ370P19
5
6  class RLE:
7      def __init__(self): #konštruktor
8          self.__escape = 255 #zapúzdrenie
9
10     def compress(self, data): #metóda s logikou RLE
11         res = [] #prázdny zoznam pre výsledky metódy
12         i = 0 #index pre začiatok
13         while i < len(data):
14             count = 1
15             while i + 1 < len(data) and data[i] == data[i+1]: #cyklus hľadá rovnaké po sebe idúce čísla
16                 count += 1
17                 i += 1
18             res.extend([self.__escape + count, data[i]]) #do výsledku sa pridá dvojica escape+počet a hodnota
19             i += 1
20         return res
21
22     def decompress(self, data): #dekompresia kompresie
23         res = []
24         for i in range(0, len(data), 2): #prechádza cez zoznam v pároch
25             count = data[i] - self.__escape #vypočet pôvodného počtu opakovaní
26             res.extend([data[i+1]] * count)
27         return res
28
29 input_file = "input.txt" #vstupné dáta
30 output_file = "output.txt" #výstupné dáta
31
32 try:
33     with open(input_file, "r") as f:
34         a = [int(x) for x in f.read().split()] #text načíta, rozdelí podľa medzier a prevedie na celé čísla
35
36     if not a: #ošetrenie ak je vstup prázdny
37         print("V súbore nie sú žiadne dáta :(")
38     else:
39         rle = RLE()
40         comp = rle.compress(a) #spustenie RLE kompresie
41         decomp = rle.decompress(comp) #spustenie dekompresie
42
43         with open(output_file, "w") as f: #vytvorenie výstupného súboru a zápis výsledkov
44             f.write("RLE kompresia s escape znakom 255:\n")
45             f.write(" ".join(map(str, comp)) + "\n")
46             f.write("Dekompresia kompresie:\n")
47             f.write(" ".join(map(str, decomp)) + "\n")
48
49         print(f"Wowooo, podarilo sa! Výsledok je uložený v súbore '{output_file}'.")
50
51 except FileNotFoundError: #ošetrenie neexistencie súboru
52     print(f"Chyba - súbor {input_file} neexistuje :(")
53 except ValueError: #ošetrenie situácie keď má súbor chybné znaky
54     print("Chyba - vstup obsahuje nesprávny formát dát (desatinné čísla alebo text) :(")
```

Problematické situácie a ich ošetrenia

- A. Problém: Ak vstupný súbor neexistuje, program by pri pokuse o načítanie spadol.
Riešenie: Pomocou *FileNotFoundException*, kde je používateľ upozornený na chýbajúci súbor.
- B. Problém: V prípade, že vstup obsahuje neplatný formát dát (napr. text alebo desatinné čísla), prevod na celé čísla zlyhá.
Riešenie: Výnimka *ValueError* s chybovým hlásením.
- C. Problém: Používateľ zadá prázdny vstupný súbor a algoritmus nebude mať čo spracovať.
Riešenie: V kóde je podmienka *if*, ktorá v tomto prípade vypíše upozornenie a kompresiu nespustí.
- D. Problém: Pri dekompresii sa zamenia dáta väčšie ako 254 za escape znak.
Riešenie: Problém je riešený fixnými dvojicami.

Alternatívne riešenia úlohy

Predovšetkým som najprv uvažovala o riešení napísania programu ako jednoduchého skriptu bez použitia OOP, potom som však implementovala čítanie s porozumením na zadanie úlohy, a riešenie zamietla. Zvažované boli aj riešenia kedy sa vytvoria dva výstupné textové súbory pre kompresiu aj dekompresiu, poprípade verzia kedy pri zistení zlého formátu časti vstupných dát (napr. text medzi celými číslami), program chybu odchyť a preskočí, každopádne obe alternatívy boli zavrhnuté z dôvodu šetrenia riadkov a snahy naučiť užívateľov zadávať správne vstupné dáta.

Vstupné dáta

Vstupné dáta by mali byť vo forme textového dokumentu (*.txt*) s názvom „*input.txt*“.
V súbore sa nachádza postupnosť celých čísel, ktoré sú buď napísane vedľa seba oddelené medzerou, alebo je každé číslo napísané na osobitnom riadku. V súbore sa nemôžu vyskytovať desatinné čísla, písmená, čiarky, bodky alebo iné špeciálne znaky.

Priložené testovacie vstupné dáta „*input.txt*“ tvoria postupnosť celých čísel oddelených medzerou. Nachádzajú sa tu prípady, kedy sa číslo vyskytuje len jeden raz, viackrát, je záporné, menšie ako 255, presne 255 alebo oveľa väčšie. Testovacie dáta sa snažia kontrolovať funkčnosť rôznych prípadov ošetrenia zlyhania programu.

Výstupné dáta

Po úspešnom prebehnutí programu sa ako výstupné dáta vytvorí nový textový súbor „*output.txt*“. V súbore sa najprv zobrazí REL kompresia vstupných dát a následne pod ňou (všetko s príslušným označením, aby sa užívateľ nezmýlil) sa objaví dekompresia kompresie vstupných dát.

Priebeh práce

Počas priebehu som si najprv iba zbežne načrtla ako by mohol program fungovať (veľmi jednoduchá štruktúra sekcií a prerozdelenia programu) a následne som sa to snažila previesť do počítača. Celý priebeh začal veľmi systematicky a v istom momente ako vždy prešiel na mierny chaos a skúšanie, ale nakoniec som došla k dúfajme funkčnej verzii. Po usúdení, že princíp programu funguje správne som najprv začala uvažovať na čom všetkom by to mohlo zlyhať a pridávala som rôzne ošetrenia a finálne som strávila neprimerane dlhý čas snahou program čo najviac skrátiť, aby bol čo najpraktickejší.

Námety na vylepšenie

Námetom na vylepšenie by určite bolo pridanie väčšej užívateľskej voľnosti, ako napríklad výberu vlastného escape znaku alebo zadávanie vstupného súboru pri spustení programu. Taktiež by sa dalo program vylepšiť aby vo vstupe nič nezmenil v prípade kedy by kompresia dosiahla nulovej úspory (napríklad postupnosť čísel bez opakovania) alebo rôznymi výpočtami pre informáciu užívateľa ako napríklad výpočet percenta ušetreného miesta vďaka kompresii.

Záverečný povzdych

Celkovo mi táto úloha prišla veľmi príjemná na spracovanie (za predpokladu, že som ju teda spracovala správne). Určite to nie je tá z náročnejších, ale spracovávala sa pekne plynule a dalo sa na nej zopakovať preberané učivo – tentokrát teda bez povzdychnutia.